

Vision par Ordinateur

TP 1

Exercice1 : Utiliser Visual Studio Code avec OpenCV

1. Installer les outils nécessaires

Installer Python : Télécharge et installe : Python

IMPORTANT pendant l'installation : coche "Add Python to PATH"

Installer VS Code : Visual Studio Code

Installer l'extension Python

Dans VS Code :

1. Clique sur **Extensions** ()
2. Recherche **Python**
3. Installe l'extension officielle de Microsoft

2. Créer ton projet

1. Crée un dossier (ex : vision_project)
2. Ouvre ce dossier dans VS Code : File → Open Folder

3. Ajouter les fichiers

Dans ton dossier :

-  test_image.py
-  image.jpg

4. Installer OpenCV

Dans le terminal VS Code : Menu → Terminal → New Terminal

Puis tape : pip install opencv-python

5. Écrire le code

Dans test_image.py :

```
import cv2
img = cv2.imread('image.jpg')
if img is None:
    print("Erreur : image non trouvée")
else:
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    cv2.imshow('Image originale', img)
    cv2.imshow('Image en gris', gray)
    cv2.imwrite('gray.jpg', gray)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

6. Exécuter le code

Méthode (terminal) : python test_image.py

Résultat attendu :

- Fenêtre 1 : image originale
- Fenêtre 2 : image en gris
- Fichier gray.jpg créé

Exercice2 : Analyse d'image

Objectif : Comprendre les propriétés d'une image

Travail demandé :

1. Charger une image
2. Convertir en gris
3. Afficher : histogramme, contours
4. Appliquer un filtre

Solution complète :

```
import cv2
import matplotlib.pyplot as plt
# Charger image
img = cv2.imread('image.jpg')
if img is None:
    print("Erreur")
else:
    # Niveaux de gris
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # Filtrage
    blur = cv2.GaussianBlur(gray, (5,5), 0)
    # Contours
    edges = cv2.Canny(blur, 100, 200)
    # Histogramme
    plt.hist(gray.ravel(), 256, [0,256])
    plt.title("Histogramme")
    plt.show()
    # Affichage
    cv2.imshow('Original', img)
    cv2.imshow('Gris', gray)
    cv2.imshow('Flou', blur)
    cv2.imshow('Contours', edges)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

Questions TP :

1. Pourquoi on convertit en niveaux de gris ?
2. Quel est l'effet du filtre Gaussian ?
3. Que représentent les contours ?
4. Que montre l'histogramme ?

Exercice3 : Acquisition et prétraitement

Objectif :

- Capturer ou charger une image
- Appliquer un prétraitement
- Sauvegarder le résultat

Travail demandé :

1. Charger une image ou capturer depuis la webcam
2. Convertir en niveaux de gris
3. Appliquer un filtre (Gaussien ou Médian)
4. Redimensionner l'image en 320×240
5. Sauvegarder l'image prétraitée

Exercice4 : Représentation et description d'images

Objectifs :

1. Charger une image couleur
2. Convertir en niveaux de gris
3. Détecter les **contours**
4. Calculer l'**histogramme**
5. Détecter des **points clés locaux (ORB)**
6. Appliquer **Fourier** et filtrage **Gabor**
7. Sauvegarder les résultats

Exercice5 : Classification binaire simple (K-NN, SVM, Softmax)

1. Installer et importer les packages

```
!pip install -q scikit-learn matplotlib
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay
```

2. Charger le dataset

```
data = load_breast_cancer()
X = data.data      # Features numériques
y = data.target    # Labels 0 (malin) ou 1 (bénin)
```

```
print("Taille du dataset :", X.shape)
print("Classes :", np.unique(y))
```

3. Split train/test

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

4. Standardisation

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

5. Classificateur K-NN

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)
print("Précision K-NN :", accuracy_score(y_test, y_pred_knn))
```

6. Classificateur SVM

```
svm = SVC(kernel='linear')
svm.fit(X_train, y_train)
y_pred_svm = svm.predict(X_test)
print("Précision SVM :", accuracy_score(y_test, y_pred_svm))
```

7. Classificateur Softmax

```
softmax = LogisticRegression(multi_class='multinomial', max_iter=1000)
softmax.fit(X_train, y_train)
y_pred_softmax = softmax.predict(X_test)
print("Précision Softmax :", accuracy_score(y_test, y_pred_softmax))
```

8. Matrices de confusion

```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_knn)
plt.title("Matrice de confusion K-NN")
plt.show()
```

```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_svm)
plt.title("Matrice de confusion SVM")
plt.show()
```

```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_softmax)
plt.title("Matrice de confusion Softmax")
plt.show()
```